

Flexible Job-Shop Scheduling Project Review

Part D Experiments and Failure Analysis

Instance generation • independent validation • scheduling • evaluation

Evidence at a glance

The checked-in Part D package contains 20 curated instances and 120 solver-instance runs: six methods per instance. All 120 schedules passed the independent C++ validator. The results are one run per solver and case with source-fixed stochastic seeds; they are descriptive evidence, not estimates of seed robustness or statistical significance.

Scope boundary

The external problem-statement file mentioned in project requests was not present in the reviewed repository. This report therefore documents the implemented code, checked-in instance files, and measured experiment data. It distinguishes implemented behavior from assumptions, missing tests, and future proposals rather than claiming unverified compliance with absent requirements.

6 October 2026

Inter-IIT Project Team

Abstract

This report records the implemented flexible job-shop model; the C++/Python generation and analysis pipeline; the independent validator and candidate evaluator; and the exact search decisions made by six scheduling methods. The results section covers instance structure, feasibility, makespan, runtime, lower-bound gap, machine utilization, and critical paths. Failure analysis separates observed outcomes from causal hypotheses and proposed experiments. A source audit identifies implementation details that materially affect behavior, including tabu aspiration ordering and the absence of a mutation operator in the memetic solver. Limits on experiment count, reproducibility, and the available problem specification are stated explicitly.

Contents

Abstract	i
List of Figures	iv
List of Tables	v
1 Purpose, Scope, and Evidence	1
1.1 Project surfaces included	1
1.2 Evidence labels	2
1.3 Document map	2
2 Problem Model, Instance Structure, and Schedule Pipeline	2
2.1 Implemented scheduling model	2
2.2 Validator contract and evaluation boundary	3
2.3 Architecture and experiment flow	3
2.4 Operation-sequence and machine-assignment encoding	3
2.5 Generator cases and schema refresh	4
2.6 Constructed workload assumptions	4
3 Generator Design and Construction Guarantees	4
3.1 Module architecture and trade-offs	5
3.2 Sampling stages and dependency structure	5
3.3 Processing-time distributions and transformations	6

3.4	Named classes	6
3.5	Validity argument, reproducibility, and limitations	8
4	Scheduling Algorithms and Implementation Decisions	8
4.1	Representation: two coupled decisions	8
4.2	The common serial decoder and evaluator	9
4.3	Shared initialization and reproducibility	10
4.4	Greedy ECT dispatch	10
4.5	Local Search	11
4.6	Simulated Annealing	11
4.7	Tabu Search	12
4.8	Genetic Algorithm	13
4.9	Memetic Algorithm	14
4.10	Cross-method comparison and interpretation	15
5	Part D Experiment Design	16
5.1	Coverage matrix	16
5.2	Hand-built edge-case catalogue	17
5.3	Protocol actually used	18
5.4	Metric definitions and interpretation	18
5.5	Budget fairness and limits	19
6	Part D Results	19
6.1	Feasibility and run completion	19
6.2	Complete per-algorithm run records	21
6.3	Per-experiment characteristics and best observed result	26
6.4	Utilization and critical path	28
6.5	Lower-bound results	28
6.6	Runtime and convergence	28
7	Part D Failure Analysis	29
7.1	Irreversible ECT commitments under contention	29
7.2	High flexibility increases routing opportunity and coupling	29

7.3	Long-duration variance and non-scaled SA temperature	30
7.4	Many very short operations expose loss of a strong incumbent	30
7.5	Weak lower bounds on shared fast alternatives	31
7.6	Flat landscapes in deterministic edge cases	31
7.7	Population methods scale their evaluation cost	32
7.8	No observed feasibility failure; remaining untested failures	32
7.9	Source-level failure analysis: behavior not isolated by the benchmark	33
8	Project Readiness and Reproducibility Gaps	34
8.1	What this report does not establish	35
9	Source-by-Source Implementation Audit	35
9.1	Generator and instance-file path	35
9.2	Shared solver representation and independent checks	38
9.3	Solver entry points	39
9.4	Experiment, visualization, and publication scripts	40
9.5	Documentation and project boundary	41
10	Reproducibility and Supplementary Definitions	42
10.1	Per-Run Data Dictionary	43
11	Algorithm Comparison	44
11.1	Aggregate results from the 20-case run	44
11.2	Trade-off by method	44
12	Recommended Solver and Decision Rule	45
12.1	Provisional selection	45
12.2	Decision rule by operating requirement	45
12.3	Final caveat	46

List of Figures

1	Observed project data flow used by the Part D runner.	3
2	Current generator module boundaries. The Python layer orchestrates and visualizes; C++ owns random sampling, model construction, validation, and serialization.	5
3	Observed C++ generation dependency order. Weight and operation position affect multiple fields; machine roles affect eligibility and processing times.	5
4	Runner data path; only independently validated schedules contribute objective diagnostics.	18

List of Tables

1	Current generator preset values assigned by <code>apply_class_preset</code>	7
2	Actual configured mechanisms and structural costs.	15
3	Part D scenario families and the stress they are intended to reveal.	16
4	Hand-built cases and their actual dimensions. Flexibility is mean eligible machines divided by machine count.	17
5	Measured makespan matrix. Every entry corresponds to an independently validated schedule. Bold marks the best value in that row; ties are all bold.	20
6	Complete per-algorithm Part D measurements. Feasibility was checked by the independent C++ validator. Utilization is averaged over all machines; case labels are shortened for layout.	22
7	Instance characteristics and best validated result per case. Runtime belongs to the fastest solver among tied best methods; case labels are shortened for layout.	27
8	Prioritized follow-up actions grounded in the reviewed code and measurements. .	34
9	Generator-side modules and consequential implementation decisions.	36
10	Solver infrastructure, validation boundary, and saved-output rules.	38
11	First-party scheduling executables.	39
12	Python and data-package modules used around the C++ components.	40
14	One-run arithmetic aggregates over the deliberately mixed 20-case suite.	44
15	Project-specific strengths and costs; not a universal ranking.	44

1 Purpose, Scope, and Evidence

This report concentrates on the project-specific experimental evidence required by Part D: generated instance families, hand-built edge cases, solver feasibility and outcomes, makespan, runtime, scale, lower-bound gap, machine utilization, critical-path behavior, observed failures, structural explanations, and concrete changes to test next. It records the current source review and the current Part D run rather than restating algorithm theory as a substitute for measurements.

What was actually executed

The C++17 generator, six solver executables, and independent validator were rebuilt from the checked-in sources with MinGW g++ using -O3. The 20 JSON files in experiments/ were each run through all six algorithms: 120 solver runs. Each output was independently passed through environment/validator.cpp; all 120 were valid. Per-run diagnostics are stored in data/part_d_results.csv. Wall-clock times are specific to this Windows host and build.

The problem statement file referenced in the initial project request was not present at the supplied repository path. This report therefore treats the checked-in code and scenario JSONs as the evidence of the implemented model. The report does not claim that its generated scenarios exactly reproduce an absent external specification.

1.1 Project surfaces included

- Generator: generator/gen_core.cpp, generator/distributions.h, and generator/fjsp_instance.h.
- Instance orchestration and visualization: generator/gen_orchestrator.py, generator/gen_visualizer.py, and algorithms/visualize.py.
- Search methods: Greedy ECT, Local Search, Simulated Annealing, Tabu Search, Genetic Algorithm, and Memetic Algorithm.
- Schedule checking: independent validator, candidate evaluator, and shared OS/machine-map decoder.
- Experiment and reporting: experiments/hand_built.py, environment/part_d_runner.py, the separate legacy environment/metrics.py, and environment/export_part_d_latex.py.
- Data: 20 experiment inputs, 76 generator outputs under instances/, 120 Part D schedules under results/part_d/, and the packaged full-precision result matrix.

1.2 Evidence labels

Label	Interpretation
Measured	Directly recorded in the 120-row Part D CSV or computed from an independently validated schedule.
Derived	Deterministic calculation from the instance and saved schedule, such as flexibility, critical path, utilization, or the lower bound defined in this report.
Hypothesis	Plausible structural explanation consistent with evidence; it is not a demonstrated causal result.
Proposed	A change to implement and compare in a future controlled experiment.

1.3 Document map

Sections 2–4 define the scheduling model, generator, and algorithm implementations. Sections 5–7 specify the experiment design, present results, and analyze failure modes using observation, evidence, hypothesis, structural explanation, and proposed improvement. Section 8 records project-readiness gaps; Section 9 audits first-party source modules; the supplementary section documents experiment reproduction and the CSV schema. The final two sections compare methods and give a provisional operating recommendation.

2 Problem Model, Instance Structure, and Schedule Pipeline

2.1 Implemented scheduling model

Each job j is an ordered chain of operations $o_{j1}, \dots, o_{jk}$. Operation o has an eligible machine set E_o and a positive processing duration p_{om} on each $m \in E_o$. A schedule chooses one eligible machine $m(o)$ and start and completion times:

$$C_o = S_o + p_{o,m(o)}, \quad m(o) \in E_o, \quad S_o \geq 0.$$

Job precedence requires $S_{j,k+1} \geq C_{j,k}$. A machine cannot process two operations at once. The reported objective is $C_{\max} = \max_o C_o$.

All jobs are available at time zero; processing is deterministic and non-preemptive; jobs may wait between operations without a buffer-capacity limit, and machines may sit idle. The code models fixed job routing/operation order with alternative machines; it does not model setup time, release dates, due dates, machine breakdowns, alternate operation routings, or uncertainty at execution time. Results apply to this model only.

2.2 Validator contract and evaluation boundary

The independent C++ validator expects a schedule JSON array with one object per operation and string `job_id`, `op_id`, and `machine_id` fields plus numeric `start_time` and `completion_time`. Its phase order checks shape, IDs/ownership, completeness and uniqueness, finite/nonnegative/forward-moving times, machine eligibility and duration, precedence in the instance’s array order, and pairwise machine non-overlap. A duration tolerance of 10^{-6} is used; interval boundaries may touch without overlap. It short-circuits deeper checks after foundational errors and reports a makespan only for a valid schedule.

The internal candidate evaluator has separate implementation and tolerances: it returns infinity for an invalid candidate, reports diagnostics, and retains a full copy of the best valid assignment. The experiment runner relies on the independent validator for feasibility status, rather than treating a solver’s own success code or evaluator score as proof of correctness. Both tools assume a parseable, structurally usable instance; malformed-instance robustness and negative-input regression coverage remain readiness work. These boundaries and model assumptions are also summarized in the companion project-assumptions note.

2.3 Architecture and experiment flow

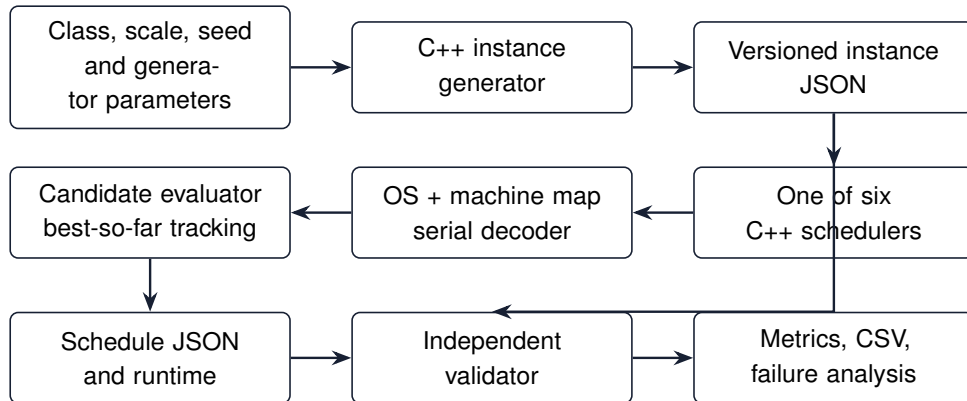


Figure 1. Observed project data flow used by the Part D runner.

2.4 Operation-sequence and machine-assignment encoding

The metaheuristics encode a chromosome/state as two linked decisions:

1. **OS**: a multiset permutation of job IDs. The k th occurrence of a job selects its k th operation, so job precedence is implicit.
2. **Machine map**: one eligible machine ID for every operation ID.

For each decoded operation, the serial schedule builder sets its start time to $\max(A_j, A_m)$, where A_j and A_m are the current job and machine availabilities, and then advances both availabilities to completion. This gives a feasible active-by-construction schedule for well-formed OS/machine-

map vectors; it does not insert an operation into an earlier idle gap. The operation lookup was changed in this review to a precomputed index, reducing decode lookup from a nested full-instance scan to expected constant-time lookup per operation. Candidate validation and per-machine timeline sorting remain paid for each evaluated candidate.

2.5 Generator cases and schema refresh

The `instances/` directory was refreshed from the current generator: 36 cases at 10 jobs/5 machines, 36 at 20 jobs/8 machines, and four standalone cases (including the 1,000-job/50-machine stress instance). The former 37 integer-ID legacy instance files were replaced by current-schema data; generator outputs now use string IDs and `machine_id/processing_time` fields. Solver outputs and charts were moved to `results/` and `analysis/`; `instances/` is reserved for generated problem instances and its catalog.

The regenerated batches use generator classes `average`, `easy`, `hard`, `extreme`, `bottleneck_heavy`, `high_flexibility`, `low_flexibility`, `unbalanced`, `high_variance`, `machine_advantage`, `large_scale`, and `small_tight`, across seeds 1, 2, and 3. The 120-run solver experiment below uses the 20 curated JSON files in `experiments/`, not all 76 generator-library outputs. This distinction avoids implying that the 72 batch instances were benchmarked against all solvers.

2.6 Constructed workload assumptions

Generated processing times are sampled, scaled, rounded, and clamped to positive integers. Each operation receives at least one eligible machine. Job length is a clamped Poisson draw. Machine roles, job-weight classes, and positional effects induce correlations. A bottleneck machine in these presets can be faster and more frequently eligible; it may therefore attract many operations and become contended. This is a fast-resource contention model, not a model of a deliberately slow bottleneck. Exact formulas, all 12 preset values, construction guarantees, and generator limitations are reported in Section 3.

The names of distributions describe the sampling mechanism before transformations; clamping/rounding changes the final empirical distribution. A seed is reproducible with the same generator/library/toolchain; standard-library distribution implementations are not guaranteed bit-identical across vendors.

3 Generator Design and Construction Guarantees

This section documents the current `gen_core_v1` implementation. It replaces historical descriptions of V1/V2/V3 source files that are not present in the current generator directory. A companion repository note records the same implementation details for developers.

3.1 Module architecture and trade-offs

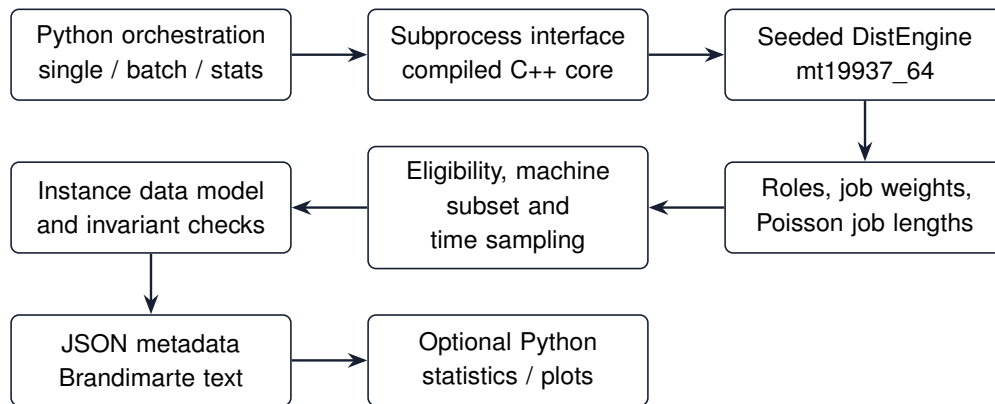


Figure 2. Current generator module boundaries. The Python layer orchestrates and visualizes; C++ owns random sampling, model construction, validation, and serialization.

- **C++ core:** fast, self-contained standard-library sampling and instance construction. Trade-off: numeric argument validation is incomplete and the generator must be compiled before use.
- **Distribution engine:** centralizes all draws on one seeded `std::mt19937_64`. Trade-off: standard-library distribution transforms can differ by vendor/version, so seed-only cross-platform bit-identical output is not guaranteed.
- **Instance model:** separates operations/jobs/metadata from the generation pipeline and checks structural invariants before output. Trade-off: JSON serialization is hand-written and needs care when adding strings or schema fields.
- **Python orchestration:** makes batch generation and statistics convenient. Plotting is optional and requires Matplotlib; it is a visualization layer, not a correctness oracle.

3.2 Sampling stages and dependency structure

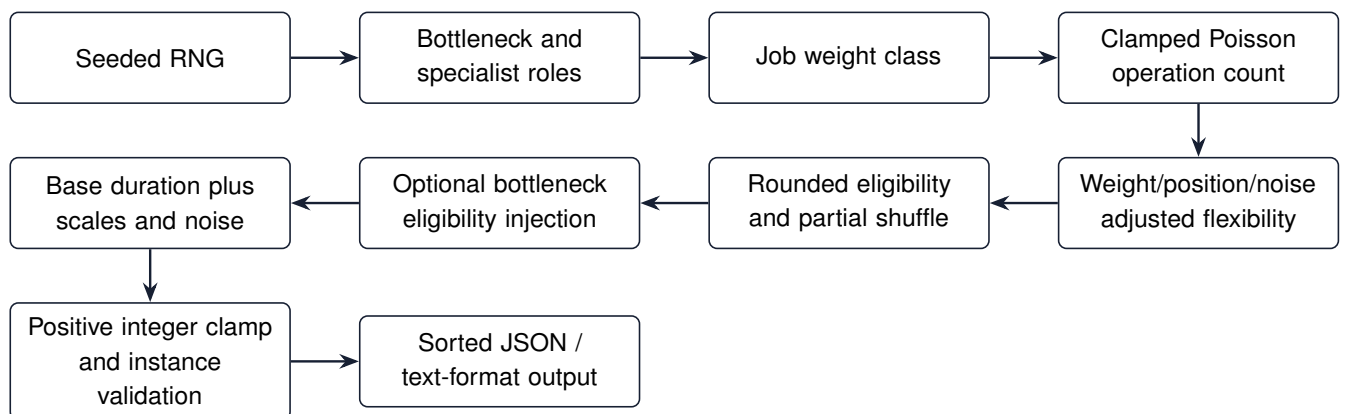


Figure 3. Observed C++ generation dependency order. Weight and operation position affect multiple fields; machine roles affect eligibility and processing times.

For job weight $w_j \in \{-1, 0, +1\}$ (light/medium/heavy), the operation-count Poisson mean is respectively 0.6λ , λ , or 1.5λ , where λ is the class parameter `ops_mean`. The Poisson result is clamped to $[1, \text{ops_max}]$. Weight probabilities are based on the configured job-weight variance and normalized by the categorical sampler.

For operation index k of q operations, the position ratio is $r = k/(q - 1)$ when $q > 1$ and zero otherwise. Effective flexibility is

$$f_{jk} = \text{clamp}(f - 0.05w_j - 0.08r + \epsilon, 0.01, 1), \quad \epsilon \sim \mathcal{N}(0, \sigma_f),$$

with class flexibility f and flexibility standard deviation σ_f . The number of eligible machines is $\min(m, \max(1, \text{round}(mf_{jk})))$. A partial Fisher–Yates shuffle selects distinct machines uniformly without replacement. A Bernoulli bottleneck event can add one selected bottleneck machine if absent.

3.3 Processing-time distributions and transformations

For $p_{\min}$ and $p_{\max}$, the base center is $c = (p_{\min} + p_{\max})/2$ and half-range is $h = (p_{\max} - p_{\min})/2$. The implemented base sampler is:

- lognormal: $\mu = \log(c)$ and $\sigma = 0.1 + 0.9v$, centered on the median, not the mean;
- normal: mean c , standard deviation $hv/2$;
- gamma: shape $\alpha = 1 + 10(1 - v)$ and scale $\beta = c/\alpha$;
- uniform: interval $[c - hv, c + hv]$.

Here v is processing-time variance control. The sampled base value is rounded and clamped to $[p_{\min}, p_{\max}]$. Each eligible-machine duration is then multiplied by the job factor $1 + 0.3w_jv_{\text{job}}$, where v_{job} is the configured job-weight variance, the positional factor $1 + 0.15r$, the bottleneck scale when applicable, the specialist scale on a per eligible-operation/machine Bernoulli event, and optional uniform noise in $[1 - \eta, 1 + \eta]$, where η is the configured noise level. The final value is rounded and clamped to $[1, p_{\max}]$.

Interpretation of “bottleneck” and processing-time bounds

The configured bottleneck role is faster, not slower, and may also be force-added to eligible sets. Its purpose is to create an attractive shared resource and contention. The base draw respects $p_{\min}$, but subsequent speed scaling can produce a final duration below $p_{\min}$; the final guarantee is a positive integer no greater than $p_{\max}$.

3.4 Named classes

Table 1 shows the current parameter values for every class; these are parameter settings, not guarantees about the realized sample statistics.

Table 1. Current generator preset values assigned by `apply_class_preset`.

Class	Ops mean	Ops max	Flex.	Flex. SD	PT range	PT law (v)	Bneck. (p, s, n)	Adv. (p, s)	Job var. / noise
average	5	10	.50	.10	1–30	lognormal (.50)	.10/.70/1	.10/.40	.30/.05
easy	3	6	.80	.05	1–20	normal (.20)	0/1/0	0/1	.10/.03
hard	7	15	.25	.15	1–60	lognormal (.80)	.30/.40/2	.25/.25	.50/.08
extreme	10	20	.15	.20	1–100	gamma (1.00)	.50/.25/3	.35/.15	.70/.12
bottleneck_heavy	5	10	.40	.10	5–40	lognormal (.50)	.70/.35/1	0/1	.30/.05
high_flexibility	5	10	.90	.05	1–30	lognormal (.50)	.10/.70/1	.20/.40	.20/.05
low_flexibility	5	10	.12	.05	1–30	lognormal (.50)	.10/.70/1	0/1	.20/.05
unbalanced	5	15	.40	.15	1–50	gamma (.70)	.20/.55/2	.10/.35	.80/.10
high_variance	5	10	.50	.10	1–100	lognormal (1.00)	.10/.70/1	.15/.30	.40/.15
machine_advantage	5	10	.50	.10	5–40	lognormal (.50)	0/1/0	.65/.18	.30/.05
large_scale	6	12	.35	.12	1–50	lognormal (.60)	.15/.55/2	.15/.35	.40/.07
small_tight	3	5	.20	.05	1–15	normal (.25)	0/1/0	0/1	.10/.03

3.5 Validity argument, reproducibility, and limitations

For valid parameter values, each job has at least one operation; every operation receives at least one distinct eligible machine in range; durations are positive integers; internal job and operation indices are contiguous; and array order defines each job's precedence chain. The generator invokes `FJSPInstance::validate()` before output. Its checks include job count and contiguous IDs, nonempty operation lists, operation/job/index consistency, nonempty machine eligibility, machine range, positive durations, duplicate machine choices, and total-operation metadata consistency. Since every operation has an eligible machine, a serial schedule is an existence proof of instance feasibility: process operations in job order, one at a time, on any eligible machine.

The proof is conditional on sane parameters. The current CLI explicitly checks positive job/machine counts and $1 \leq p_{\min} \leq p_{\max}$, but does not comprehensively validate all probabilities, variance ranges, or cross-field constraints. This is a development-readiness gap rather than a property established by the construction proof. JSON serialization is hand-written, and the implementation does not promise bit-identical random distribution output across C++ library vendors. Those limitations should be addressed with systematic range checks, serialization tests, and a manifest that records compiler/library versions.

The seed and complete preset parameters are written to JSON metadata. A `-class` preset replaces class-level structural values after parsing; job count, machine count, and seed remain separate. Specialist machines are a random subset of $\max(1, \text{round}(0.3m))$ when advantage probability is positive, and none otherwise; bottleneck subset size is clamped to $[0, m]$ and is selected only when bottleneck probability is positive. The roles can overlap. Parameter-based mode requires the core parameters explicitly and fills optional extended parameters with defaults. The report reproducibility section gives the batch commands; the repository also contains a developer-facing Markdown version of this design.

4 Scheduling Algorithms and Implementation Decisions

4.1 Representation: two coupled decisions

All six solvers ultimately submit the same object to the evaluator: one explicit machine, start time, and completion time for each operation. The five search-based solvers do not search directly over those times. They search over two linked structures:

Operation sequence (OS)

A vector containing each job ID once for every operation in that job. The k th occurrence of job j means the k th operation of that job. This multiset encoding preserves job precedence without storing a separate precedence graph in every candidate.

Machine-assignment map

A map from operation ID to one eligible machine ID. One assignment is stored per operation, independently of the OS order. The source names this map `ma`; this report reserves MA for the Memetic Algorithm.

The OS determines which job operation is decoded at each position; the machine-assignment map determines its processing duration and the machine queue to which it is appended. Assignment and sequencing therefore interact: moving an operation to a different machine changes both its duration and its resource conflicts. The code uses string IDs and hash maps for these structures. This keeps the JSON boundary readable, but copying and hashing candidates has higher constant cost than a compact integer-index representation.

4.2 The common serial decoder and evaluator

The shared decoder in `algorithms/fjsp_algorithm.h` first maps the OS vector to operation IDs by maintaining the next-operation position for each job. It then traverses that operation sequence once. If the next operation is o of job j , assigned to machine m , with processing time p_{om} , its start and completion are:

$$S_o = \max(A_j, A_m), \quad C_o = S_o + p_{om},$$

where A_j and A_m are the current availability times of the job and machine. Both availabilities are advanced to C_o .

What this decoder guarantees—and what it does not

For a well-formed OS with the correct multiplicity and a valid machine-assignment map, the decoder emits a complete schedule that respects job precedence, uses eligible machines, and does not overlap work on any machine. It commits each operation at the first time both resources are available. It does not insert that operation into an earlier idle gap left by a previous decoding decision. Thus the search is over schedules representable by this serial construction, not over every possible feasible schedule.

The evaluator in `environment/evaluator.cpp` checks candidate completeness, duplicate and unknown IDs, job ownership, finite and non-negative times, machine eligibility, duration, precedence, and machine overlap. Invalid candidates return infinity and write diagnostics to standard error. A valid candidate updates the saved best only when its makespan is *strictly* smaller; equal makespans do not replace the first stored best. Only the best candidate is retained by the evaluator, but each search method may additionally copy its current state, neighbors, population, or tabu records. Export sorts the saved assignments by start time, then by operation ID for ties.

The decoder performs expected $O(N)$ work for N operations using hash-map lookups. A full evaluator call builds candidate and machine indexes, checks operations and precedence, and sorts machine timelines; its worst-case candidate cost is $O(N \log N)$. The candidate is fully rebuilt and checked after every proposal. No solver uses an incremental objective or partial schedule update.

4.3 Shared initialization and reproducibility

Local Search, Simulated Annealing, Tabu Search, Genetic Algorithm, and Memetic Algorithm initialize OS by repeating each job ID according to its operation count and shuffling the complete vector with `std::mt19937` seeded to 42. Each operation is assigned a randomly selected eligible machine. There is no ECT/greedy seed, warm start, CLI seed override, or restart. The generated schedule is evaluated before search begins.

The seed is fixed in source, not supplied through the command line. For a given C++ library and executable, the stochastic procedures are repeatable; this does not make the experiment a multi-seed study. Iteration, generation, neighborhood, and population sizes are also fixed at each executable's call site. The six budgets differ substantially and do not represent equal evaluation counts or equal runtime.

4.4 Greedy ECT dispatch

Greedy ECT

One pass of earliest-completion dispatch; no search state or random generator.

Candidate set. At each loop, the solver considers exactly the next unscheduled operation of every unfinished job. For each such operation, it tests every eligible machine.

Decision rule. For candidate pair (o, m) it computes $\max(A_j, A_m) + p_{om}$ and chooses the pair with the smallest resulting completion time. It then commits that operation, advances the job's operation index and availability, and advances the selected machine's availability. This continues until every job is complete.

Name versus behavior. The executable is called `greedy_spt`, but the implementation minimizes earliest completion time (ECT); it does not globally sort operations by shortest processing time. Although a source comment mentions processing time as a possible tie-break, the code only replaces the incumbent when ECT is strictly smaller. Exact ties therefore keep whichever candidate was encountered first; eligible machines are stored in an unordered map, so no stable tie policy is promised.

Cost. One schedule is constructed in N dispatch rounds. Each round scans up to J ready jobs and their eligible alternatives; a coarse bound is $O(NJ\bar{e})$ for J jobs and mean eligibility width $\bar{e}$. The method uses availability state and the output schedule, not a population of candidates.

Strengths. Very low overhead; it supplies a useful feasible baseline and is deterministic except for implementation-dependent ordering on exact ties.

Costs and risks. Each assignment is irreversible. ECT has no downstream look-ahead, contention estimate, backtracking, or subsequent improvement pass. The choice that finishes the current operation earliest can make a popular machine the queue for later critical work.

4.5 Local Search

Local Search (LS)

A single current schedule is perturbed repeatedly; only non-worsening moves are accepted.

Initial state. One random OS/machine-map candidate is generated and evaluated.

Neighborhood. Each of the 10,000 iterations copies the current OS and machine-assignment map. A fair coin selects one of two move families:

1. swap two uniformly selected positions in OS; or
2. choose a job uniformly, then an operation uniformly within that job, and change its machine to a uniformly selected different eligible machine.

The first move can select the same position twice or swap two equal job IDs, which leaves OS unchanged. The second can select a fixed-routing operation, in which case no assignment changes. These are valid but potentially no-op proposals.

Acceptance and memory. A valid neighbor replaces the current state when its makespan is less than or equal to the current makespan. Equal-cost moves are accepted, so the trajectory can move across a plateau. The evaluator records a strict best separately, which prevents equal-cost plateau moves from replacing the saved best schedule.

Termination and cost. Exactly 10,000 proposals are attempted. Each proposal is decoded and fully evaluated, including proposals that made no effective change. There is no stagnation stop, restart, greedy initialization, or progress log.

Strengths. Simple, low-memory intensification. Accepting ties permits movement through flat regions.

Costs and risks. It never accepts a worsening move, so it can be trapped at a local minimum. Random swaps are not guaranteed to change the decoded operation order, and uniformly selected operations need not lie on a critical path.

4.6 Simulated Annealing

Simulated Annealing (SA)

Local Search's single-move neighborhood with a temperature-controlled probability of accepting a worse score.

Neighborhood. The same 50/50 OS-swap versus machine-map-change choice and the same possible no-op cases as LS.

Acceptance. Let $\Delta = C_{\max}^{\text{neighbor}} - C_{\max}^{\text{current}}$. The solver accepts every valid neighbor with $\Delta \leq 0$. For $\Delta > 0$, it accepts when a uniform random draw is below $\exp(-\Delta/T)$. The temperature is multiplied by 0.999 after each iteration, starting at 100. The call site requests 20,000 iterations.

State and best. Accepted uphill moves update the current state and can make the current score worse; the evaluator's independently retained strict best remains the exported answer. No acceptance-rate feedback, reheating, temperature calibration from observed deltas, or

best-so-far curve is implemented.

Cost. One complete candidate evaluation per iteration, or 20,000 proposals, in addition to the initial evaluation. The geometric schedule reduces temperature to approximately 2.1×10^{-7} by the final iteration.

Strengths. Uphill acceptance can leave a local basin without requiring population memory; it had a strong quality/runtime balance in this one configured experiment.

Costs and risks. The fixed absolute temperature is scale-sensitive. The fixed geometric cooling schedule can become effectively frozen before useful moves are found on instances with large objective deltas; the code does not measure whether that happened.

4.7 Tabu Search

Tabu Search (TS)

At each iteration, sample several one-move neighbors and select the best candidate not currently marked tabu.

Neighborhood and budget. Each of 2,000 iterations independently generates 20 neighbors using the same OS/machine-map move families as LS and SA. Sampling is with replacement in effect: two proposals can represent the same state. The method evaluates every sampled candidate before choosing a move, for up to 40,000 full candidate evaluations.

Admissibility and choice. It selects the lowest-makespan sampled neighbor whose serialized full state is absent from the tabu set. A candidate whose makespan is a new global best is intended to pass an aspiration test. The tabu record is the whole OS plus the whole machine-assignment map, not a changed-move signature. A FIFO queue with tenure 100 expires old state strings. If all 20 sampled candidates are tabu, no move is made during that iteration.

Source-level aspiration defect. In the current call sequence, `eval.evaluate(neighbor)` updates the evaluator's global best before the solver compares the neighbor's score with `eval.get_best_objective()`. Therefore a newly best candidate has already made the two values equal when the aspiration predicate is computed; the strict comparison is false. The implemented aspiration exception cannot admit a tabu candidate on the basis of that new best. This is a code-derived finding, not an experimentally isolated cause of any reported score.

State-key caveat. The serialized machine-assignment portion iterates an `unordered_map` without sorting keys. The key is therefore not a canonical encoding of a logical assignment across implementations. In addition, when an expired key is erased from the set, an older duplicate queue entry can erase a key that was reinserted more recently.

Termination and cost. The run always uses tenure 100, 2,000 iterations, and neighborhood size 20. There is no fallback move, tabu-hit counter, cycle diagnostic, or convergence log.

Strengths. Candidate comparison per iteration offers more local alternatives than one-move hill climbing and can accept worsening moves when they are the best admissible neighbor.

Costs and risks. Highest observed mean runtime in the reported matrix. Full-state serialization and hashing add work; the no-admissible-neighbor case stalls for that iteration; the aspiration test does not behave as intended.

4.8 Genetic Algorithm

Genetic Algorithm (GA)

A population of OS/machine-map chromosomes is repeatedly selected, recombined, mutated, and evaluated.

Chromosome and score. Each member holds an OS vector, machine-assignment map, makespan, and fitness. A valid candidate receives fitness $10000/C_{\max}$; an invalid candidate receives zero. This is a monotone transformation of makespan, so the ranking is preserved for positive makespans. Initial population size is 50 and all members are random.

Parent selection. A size-two tournament samples two population indexes uniformly and returns the member with greater fitness. The two indexes can match; on equal fitness the conditional returns the second sampled member. There is no rank scaling, roulette selection, or diversity adjustment.

Recombination. Crossover is applied to every parent pair:

- Machine-map crossover uses independent uniform per-operation exchange: with probability $1/2$, the two children exchange that operation's assigned machine. Because both parents use an eligible machine at that operation, exchanging the values preserves eligibility.
- OS uses job-based crossover. A random subset of job IDs is selected. For child 1, selected-job entries retain parent 1's relative order and other positions are filled in parent 2's order; child 2 uses the complementary construction. Job multiplicities are preserved, and the decoder maps each occurrence to that job's next operation.

Mutation. Independently for each child, OS mutation occurs with probability 0.1 and swaps two random positions. Machine-map mutation occurs with probability 0.1; it selects a job uniformly, then one of that job's operations uniformly, and assigns a different eligible machine uniformly when an alternative exists. These are probabilities per child and per chromosome field, not per gene.

Replacement and stopping. One maximum-fitness member is copied as an elite into each next generation. Parent pairs are then crossed and mutated; offspring are evaluated until the population returns to size 50. The call site runs 200 generations. There is no convergence-based stop, restart, adaptive mutation, explicit diversity measurement, or ECT seed.

Evaluation count. Initial population costs 50 evaluations. Each generation keeps one elite and generates 49 new members, for up to 9,850 offspring evaluations over 200 generations, approximately 9,900 total evaluations before any additional CLI/report overhead.

Strengths. Maintains multiple candidate regions and uses different recombination for assignment and order.

Costs and risks. Random initial candidates can be weak. Recombination preserves chromosome structure but does not preserve machine-load balance or good schedule blocks. Fixed population and generation counts do not guarantee useful diversity or equal work to other methods.

4.9 Memetic Algorithm

Memetic Algorithm (MA)

The same population framework is combined with strict-improvement local polishing.

Population and selection. It uses population 30, the same random initialization, size-two tournament selection, one-member elitism, uniform per-operation machine-map crossover, and job-based OS crossover as GA.

A consequential difference from GA. The current `memetic_algorithm.cpp` has no GA-style mutation function or mutation call. Its variation comes from crossover and local-search proposals only. It must not be described as using all GA operators.

Local polishing. Each of the 30 random initial chromosomes receives 50 local-search proposals. Each offspring receives 10 proposals. A proposal uses the same 50/50 OS-swap or machine-map-change move family, but polishing accepts only a valid *strict* improvement. Unlike the standalone LS solver, it does not accept equal-score plateau moves. The offspring is evaluated before polishing; every proposal also incurs a complete decode/evaluation.

Generation formation. One best chromosome is copied to the next population. Parent pairs are crossed; child 1 and, if a slot remains, child 2 are evaluated and polished before insertion. With a population of 30, this creates 29 new members per generation. The executable runs 100 generations.

Approximate evaluation count. Initialization requires 30 initial evaluations and 30×50 polishing evaluations. Each generation creates 29 offspring, each with one evaluation before polishing and 10 proposal evaluations: $100 \times 29 \times 11$ further evaluations. This is about 33,430 complete candidate evaluations, ignoring any invalid-candidate differences. It is substantially more work than the GA's approximately 9,900, so their raw outcomes do not isolate the value of the hybrid design.

Strengths. Adds local intensification after global recombination and keeps the best-ever solution through evaluator-level storage and population elitism.

Costs and risks. The local phase rejects uphill and neutral moves, so it retains hill-climbing traps. No mutation may reduce exploration after the population converges; there is no diversity monitor or selective polishing budget.

4.10 Cross-method comparison and interpretation

Table 2. Actual configured mechanisms and structural costs.

Method	Search state and acceptance	Configured work	Principal implementation risk
Greedy ECT	One schedule; commit the current best ready operation-machine pair.	One dispatch loop; scans ready jobs and eligible choices.	Irreversible choices; no stable exact-tie policy.
LS	One schedule; accept valid $\Delta \leq 0$.	10,000 one-neighbor proposals.	Local trapping; proposals can be ineffective.
SA	One schedule; accept $\Delta \leq 0$ or probabilistically accept $\Delta > 0$.	20,000 one-neighbor proposals; $T_0 = 100$, $\alpha = 0.999$.	Fixed temperature scale and cooling; no progress telemetry.
TS	One schedule; choose the best admissible of 20 proposals.	2,000 iterations; at most 40,000 neighbor evaluations; tenure 100.	Ineffective aspiration check, noncanonical keys, and no fallback.
GA	Population; size-two tournament, crossover, field-level mutation, one elite.	50 members; 200 generations; approximately 9,900 evaluations.	Random starts and no diversity/adaptation controls.
MA	Population; crossover, one elite, strict-improvement polishing.	30 members; 100 generations; approximately 33,430 evaluations.	No chromosome mutation; much larger evaluation budget than GA.

The evaluator's strict best update and each search method's current-state acceptance are distinct mechanisms. A method can accept a neutral or uphill state without losing the already saved strict best. This separation explains why the output file represents the best valid candidate *encountered*, not necessarily the final state at termination.

Runtime and final makespan compare the configured executables, not algorithm families at equal computational effort. Evaluation counts, full validation, string-based data structures, and algorithm-specific copying all contribute to measured cost. No solver currently records per-iteration best score, accepted-move count, diversity, or evaluator-call trace.

5 Part D Experiment Design

5.1 Coverage matrix

The Part D matrix contains the curated generated classes plus all ten hand-built edge cases. The purpose is structural coverage rather than drawing a random sample from one broad distribution.

Table 3. Part D scenario families and the stress they are intended to reveal.

Instance type	Purpose and targeted scheduling structure	Cases
Average	Typical generated instance; checks ordinary assignment/sequencing behavior.	1
Low flexibility	Few alternatives; emphasizes precedence and constrained routing.	1
High flexibility	Many alternatives; emphasizes machine-selection quality and contention.	1
Bottleneck	Operations compete for an attractive, frequently eligible machine.	2
Balanced	Checks behavior under a less deliberately skewed workload.	1
High variance	Short and long processing-time alternatives coexist.	1
Machine advantage	Tests whether the solver exploits machine specialization.	2
Large / small	Measures scaling against a compact case where lower bounds are informative.	2
Extreme	Exposes interaction of low flexibility, high duration spread, and machine effects.	1
Single machine	Removes assignment flexibility and isolates machine sequencing.	1
One job	Removes inter-job machine contention and isolates precedence chains.	1
Extreme time gaps	Contrasts 1-unit and 10,000-unit alternatives.	1
Near-total / near-zero flexibility	Separates assignment-dominated from nearly fixed-routing cases.	2
Identical processing times	Creates symmetric choices and many objective ties.	1
Long critical chains	Makes precedence dominate the makespan lower bound.	1
Many very short operations	Stresses sequencing and search overhead relative to duration.	1

5.2 Hand-built edge-case catalogue

Table 4. Hand-built cases and their actual dimensions. Flexibility is mean eligible machines divided by machine count.

Case	Jobs	Machines	Operations	Construction / expected diagnostic
single_machine	5	1	10	All work on one machine; detects unnecessary search when routing is fixed.
one_job	1	5	10	One 10-operation chain with full eligibility; objective is precedence-dominated.
bottleneck_hand	10	5	30	Each job has one 50-unit operation forced onto machine 3; isolates unavoidable contention.
extreme_time_gaps	3	2	9	Each operation offers a fast 1-unit and slow 10,000-unit alternative.
machine_advantage_hand	2	2	6	Jobs have opposite machine preference (1 versus 100 time units).
near_total_flexibility	3	3	9	All operations can use all machines at equal durations.
near_zero_flexibility	3	3	9	Each operation has one eligible machine; routing is fixed.
identical_processing_times	5	5	20	Equal durations on all eligible machines produce a symmetric objective landscape.
long_critical_chains	2	5	40	Two 20-operation chains; the 100-unit job bound dominates.
many_very_short_operations	3	3	150	1-unit operations; stresses chromosome and evaluator overhead.

5.3 Protocol actually used

1. Compile all solver sources and the independent validator using the same C++17 MinGW compiler and -O3.
2. For each of 20 input JSONs, run each of six solver executables once. Stochastic algorithms use the source-level default seed 42; it is not exposed as a CLI option.
3. Remove the target schedule before starting a solver. Record solver wall-clock duration with Python's monotonic performance clock. Timeout is 180 seconds per solver and per validator.
4. Validate each schedule using the separately built C++ validator, then independently derive the reported metrics from the instance and schedule.
5. Save a row even on failure. A failed solver/validator has `feasible=false`, no makespan, and a diagnostic log. No convergence trace is synthesized from a final-only schedule.

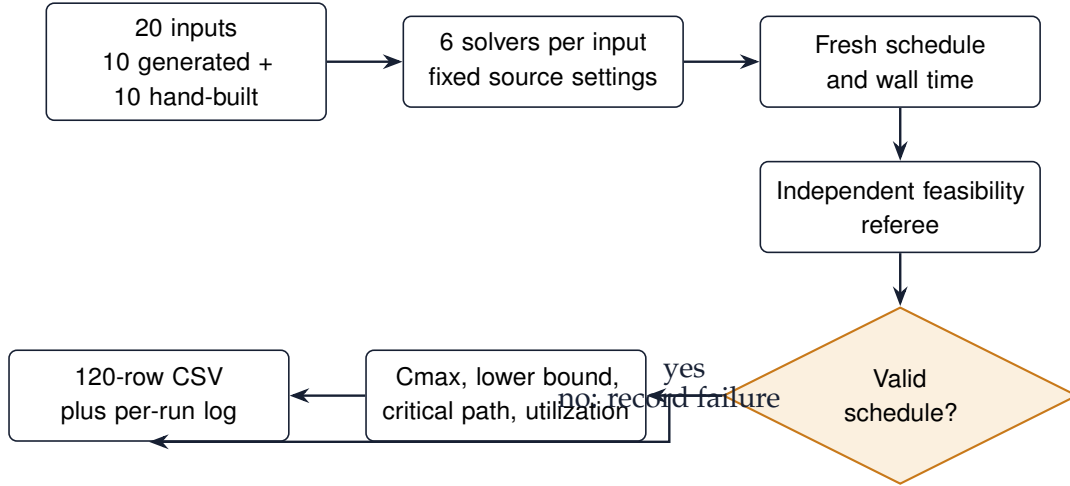


Figure 4. Runner data path; only independently validated schedules contribute objective diagnostics.

5.4 Metric definitions and interpretation

Feasibility is the independent validator's binary result. **Makespan** is the maximum completion time in a valid schedule. **Mean flexibility** is the mean eligible-machine count divided by total machine count. **Runtime** is solver invocation wall time, excluding the independent validator.

We report a deliberately transparent lower bound:

$$LB = \max \left\{ \max_j \sum_{o \in j} \min_{m \in E_o} p_{om}, \max_m \sum_{o: E_o = \{m\}} p_{om} \right\}.$$

The first term is the minimum possible serial workload of a job; the second is forced load on machines that are the sole choice for an operation. Both are valid lower bounds, but this bound can be weak when many operations share the same attractive alternative. Gap is $100(C_{\max} - LB)/LB$. Equality between a feasible makespan and this lower bound certifies optimality; a positive gap does not imply suboptimality.

Critical path is recomputed on the realized schedule’s disjunctive graph: job-precedence arcs plus consecutive operations in each machine’s scheduled order, weighted by processing duration. **Machine utilization** is busy time divided by makespan; the mean includes every named machine in the instance, including idle machines. All feasible schedules in this run had a critical path length equal to makespan, as expected for the active serial schedules produced here.

Convergence is unavailable: current executables retain/export a best schedule but do not emit iteration-wise best makespan, acceptance, diversity, or evaluation-count traces. The runner marks this explicitly as not instrumented.

5.5 Budget fairness and limits

The parameters used are those compiled into the solver sources: LS 10,000 moves; SA 20,000 moves with $(T_0, \alpha) = (100, 0.999)$; TS 2,000 iterations times neighborhood size 20 and tenure 100; GA population 50 over 200 generations; MA population 30 over 100 generations, 50 initial local-search trials per individual and 10 per offspring. These settings do not equalize candidate evaluation counts or wall time. Results are one run per cell on one host and one fixed seed; descriptive observations must not be presented as statistical significance or seed robustness.

6 Part D Results

6.1 Feasibility and run completion

All 120 solver outputs passed the independent validator: 20/20 instances had six feasible schedules. There were no solver timeouts or invalid schedules in this run. This is evidence that the current decoder and these current instances are mutually compatible; it is not evidence that malformed external input is safely handled or that every possible instance can be solved.

The experiment outcomes, including every algorithm’s makespan for every case, are summarized in Table 5. The full per-algorithm diagnostics, runtime, processing-time range, lower-bound gap, critical path IDs, utilization, and machine-role fields are in `data/part_d_run_matrix.csv`.

Table 5. Measured makespan matrix. Every entry corresponds to an independently validated schedule. Bold marks the best value in that row; ties are all bold.

Experiment	Greedy ECT	LS	SA	TS	GA	Memetic
Average	242	210	203	211	231	222
Balanced	494	339	336	329	351	342
Bottleneck (generated)	331	244	260	246	256	263
Bottleneck (hand-built)	510	510	510	510	510	510
Extreme	1375	1130	1087	1087	1130	1087
Extreme time gaps	9	9	9	9	9	9
High flexibility	176	159	160	147	191	177
High variance	820	697	586	607	680	638
Identical processing times	40	40	40	40	40	40
Large (50 jobs)	339	343	323	323	434	426
Long critical chains	100	100	100	100	100	100
Low flexibility	499	419	419	419	419	419
Machine advantage (generated)	326	229	226	229	241	229
Machine advantage (hand-built)	3	3	3	3	3	3
Many very short operations	50	58	56	58	62	63
Near-total flexibility	30	30	30	30	30	30
Near-zero flexibility	30	30	30	30	30	30
One job	50	50	50	50	50	50
Single machine	125	125	125	125	125	125
Small	83	83	83	83	83	83

6.2 Complete per-algorithm run records

Table 6 expands the matrix to every algorithm-instance run and reports size, processing-time range, feasibility, makespan, solver runtime, lower-bound gap, critical-path length, mean machine utilization, and maximum machine utilization. This avoids hiding slower or weaker methods behind the best-per-instance summary. The supplementary CSV also retains processing-time mean and deviation, bottleneck/specialist machine IDs, the full critical-path operation sequence, run purpose, algorithm seed disclosure, schedule path, convergence-trace status, and diagnostics.

Table 6. Complete per-algorithm Part D measurements. Feasibility was checked by the independent C++ validator. Utilization is averaged over all machines; case labels are shortened for layout.

Case	J	M	N	$p_{\min}/p_{\max}$	Solver	Valid	$C_{\max}$	Time (s)	Gap (%)	CP	$\bar{U}$ (%)	$U_{\max}$ (%)
average	15	5	83	3/30	ECT	Y	242.000000	0.039238	132.692308	242.000000	79.586777	90.909091
average	15	5	83	3/30	LS	Y	210.000000	0.640959	101.923077	210.000000	97.714286	100.000000
average	15	5	83	3/30	SA	Y	203.000000	1.167044	95.192308	203.000000	98.029557	100.000000
average	15	5	83	3/30	TS	Y	211.000000	2.794969	102.884615	211.000000	97.251185	100.000000
average	15	5	83	3/30	GA	Y	231.000000	1.086239	122.115385	231.000000	98.441558	100.000000
average	15	5	83	3/30	Memetic	Y	222.000000	2.381551	113.461538	222.000000	97.027027	99.549550
balanced	15	5	92	3/50	ECT	Y	494.000000	0.037412	79.636364	494.000000	63.765182	82.186235
balanced	15	5	92	3/50	LS	Y	339.000000	0.727935	23.272727	339.000000	97.227139	100.000000
balanced	15	5	92	3/50	SA	Y	336.000000	1.315081	22.181818	336.000000	97.440476	100.000000
balanced	15	5	92	3/50	TS	Y	329.000000	3.290508	19.636364	329.000000	97.446809	99.392097
balanced	15	5	92	3/50	GA	Y	351.000000	1.170256	27.636364	351.000000	93.789174	100.000000
balanced	15	5	92	3/50	Memetic	Y	342.000000	2.453237	24.363636	342.000000	95.730994	100.000000
bottleneck	15	5	74	2/40	ECT	Y	331.000000	0.036349	108.176101	331.000000	62.296073	89.728097
bottleneck	15	5	74	2/40	LS	Y	244.000000	0.704789	53.459119	244.000000	96.147541	100.000000
bottleneck	15	5	74	2/40	SA	Y	260.000000	1.112982	63.522013	260.000000	95.307692	100.000000
bottleneck	15	5	74	2/40	TS	Y	246.000000	2.752827	54.716981	246.000000	96.422764	100.000000
bottleneck	15	5	74	2/40	GA	Y	256.000000	1.116248	61.006289	256.000000	92.421875	98.828125
bottleneck	15	5	74	2/40	Memetic	Y	263.000000	2.100549	65.408805	263.000000	96.425856	98.859316
Bottleneck hand	10	5	30	5/50	ECT	Y	510.000000	0.042909	2.000000	510.000000	23.529412	98.039216
Bottleneck hand	10	5	30	5/50	LS	Y	510.000000	0.287183	2.000000	510.000000	23.529412	98.039216
Bottleneck hand	10	5	30	5/50	SA	Y	510.000000	0.551843	2.000000	510.000000	23.529412	98.039216
Bottleneck hand	10	5	30	5/50	TS	Y	510.000000	1.132391	2.000000	510.000000	23.529412	98.039216
Bottleneck hand	10	5	30	5/50	GA	Y	510.000000	0.445609	2.000000	510.000000	23.529412	98.039216
Bottleneck hand	10	5	30	5/50	Memetic	Y	510.000000	0.984195	2.000000	510.000000	23.529412	98.039216
extreme	20	5	200	1/100	ECT	Y	1375.000000	0.036061	26.494940	1375.000000	46.036364	88.800000
extreme	20	5	200	1/100	LS	Y	1130.000000	1.525314	3.955842	1130.000000	62.070796	100.000000
extreme	20	5	200	1/100	SA	Y	1087.000000	2.956778	0.000000	1087.000000	65.906164	100.000000
extreme	20	5	200	1/100	TS	Y	1087.000000	6.620623	0.000000	1087.000000	66.715731	100.000000
extreme	20	5	200	1/100	GA	Y	1130.000000	2.838816	3.955842	1130.000000	61.628319	100.000000
extreme	20	5	200	1/100	Memetic	Y	1087.000000	5.429799	0.000000	1087.000000	67.028519	100.000000
Time gaps	3	2	9	1/10000	ECT	Y	9.000000	0.031639	200.000000	9.000000	50.000000	100.000000
Time gaps	3	2	9	1/10000	LS	Y	9.000000	0.122531	200.000000	9.000000	50.000000	100.000000
Time gaps	3	2	9	1/10000	SA	Y	9.000000	0.199655	200.000000	9.000000	50.000000	100.000000

Case	J	M	N	$p_{\min}/p_{\max}$	Solver	Valid	$C_{\max}$	Time (s)	Gap (%)	CP	$\bar{U}$ (%)	$U_{\max}$ (%)
Time gaps	3	2	9	1/10000	TS	Y	9.000000	0.413966	200.000000	9.000000	50.000000	100.000000
Time gaps	3	2	9	1/10000	GA	Y	9.000000	0.147793	200.000000	9.000000	50.000000	100.000000
Time gaps	3	2	9	1/10000	Memetic	Y	9.000000	0.497468	200.000000	9.000000	50.000000	100.000000
High flexibility	15	5	75	2/30	ECT	Y	176.000000	0.032438	72.549020	176.000000	74.659091	86.931818
High flexibility	15	5	75	2/30	LS	Y	159.000000	0.622810	55.882353	159.000000	94.591195	99.371069
High flexibility	15	5	75	2/30	SA	Y	160.000000	1.207716	56.862745	160.000000	95.000000	99.375000
High flexibility	15	5	75	2/30	TS	Y	147.000000	2.733233	44.117647	147.000000	94.421769	98.639456
High flexibility	15	5	75	2/30	GA	Y	191.000000	1.065442	87.254902	191.000000	90.680628	97.905759
High flexibility	15	5	75	2/30	Memetic	Y	177.000000	2.225434	73.529412	177.000000	92.090395	99.435028
High variance	15	5	83	3/100	ECT	Y	820.000000	0.034786	144.047619	820.000000	64.512195	70.487805
High variance	15	5	83	3/100	LS	Y	697.000000	0.618459	107.440476	697.000000	98.192253	100.000000
High variance	15	5	83	3/100	SA	Y	586.000000	1.228205	74.404762	586.000000	98.020478	100.000000
High variance	15	5	83	3/100	TS	Y	607.000000	3.045876	80.654762	607.000000	93.607908	99.505766
High variance	15	5	83	3/100	GA	Y	680.000000	1.032431	102.380952	680.000000	96.823529	98.676471
High variance	15	5	83	3/100	Memetic	Y	638.000000	2.334097	89.880952	638.000000	93.981191	99.529781
Identical times	5	5	20	10/10	ECT	Y	40.000000	0.032362	0.000000	40.000000	100.000000	100.000000
Identical times	5	5	20	10/10	LS	Y	40.000000	0.242063	0.000000	40.000000	100.000000	100.000000
Identical times	5	5	20	10/10	SA	Y	40.000000	0.448039	0.000000	40.000000	100.000000	100.000000
Identical times	5	5	20	10/10	TS	Y	40.000000	1.075012	0.000000	40.000000	100.000000	100.000000
Identical times	5	5	20	10/10	GA	Y	40.000000	0.349896	0.000000	40.000000	100.000000	100.000000
Identical times	5	5	20	10/10	Memetic	Y	40.000000	0.813234	0.000000	40.000000	100.000000	100.000000
large	50	10	242	2/30	ECT	Y	339.000000	0.043356	208.181818	339.000000	70.471976	84.070796
large	50	10	242	2/30	LS	Y	343.000000	1.905812	211.818182	343.000000	93.527697	99.708455
large	50	10	242	2/30	SA	Y	323.000000	3.831360	193.636364	323.000000	96.904025	100.000000
large	50	10	242	2/30	TS	Y	323.000000	8.781932	193.636364	323.000000	95.170279	99.690402
large	50	10	242	2/30	GA	Y	434.000000	3.819780	294.545455	434.000000	86.451613	99.078341
large	50	10	242	2/30	Memetic	Y	426.000000	7.302228	287.272727	426.000000	87.816901	98.122066
Long chains	2	5	40	5/5	ECT	Y	100.000000	0.033939	0.000000	100.000000	40.000000	100.000000
Long chains	2	5	40	5/5	LS	Y	100.000000	0.329561	0.000000	100.000000	40.000000	60.000000
Long chains	2	5	40	5/5	SA	Y	100.000000	0.659998	0.000000	100.000000	40.000000	55.000000
Long chains	2	5	40	5/5	TS	Y	100.000000	1.422381	0.000000	100.000000	40.000000	50.000000
Long chains	2	5	40	5/5	GA	Y	100.000000	0.443169	0.000000	100.000000	40.000000	50.000000
Long chains	2	5	40	5/5	Memetic	Y	100.000000	1.083000	0.000000	100.000000	40.000000	55.000000
low_flexibility	15	5	81	4/30	ECT	Y	499.000000	0.039662	19.093079	499.000000	50.981964	83.967936
low_flexibility	15	5	81	4/30	LS	Y	419.000000	0.625074	0.000000	419.000000	60.954654	100.000000
low_flexibility	15	5	81	4/30	SA	Y	419.000000	1.177318	0.000000	419.000000	60.715990	100.000000
low_flexibility	15	5	81	4/30	TS	Y	419.000000	2.826194	0.000000	419.000000	60.954654	100.000000

Case	J	M	N	$p_{\min}/p_{\max}$	Solver	Valid	$C_{\max}$	Time (s)	Gap (%)	CP	$\bar{U}$ (%)	$U_{\max}$ (%)
low_flexibility	15	5	81	4/30	GA	Y	419.000000	1.103341	0.000000	419.000000	60.954654	100.000000
low_flexibility	15	5	81	4/30	Memetic	Y	419.000000	2.211097	0.000000	419.000000	60.954654	100.000000
machine_advantage	15	5	73	2/40	ECT	Y	326.000000	0.035546	44.247788	326.000000	56.748466	68.098160
machine_advantage	15	5	73	2/40	LS	Y	229.000000	0.651948	1.327434	229.000000	90.131004	94.759825
machine_advantage	15	5	73	2/40	SA	Y	226.000000	1.108441	0.000000	226.000000	89.823009	96.902655
machine_advantage	15	5	73	2/40	TS	Y	229.000000	2.695072	1.327434	229.000000	90.305677	96.943231
machine_advantage	15	5	73	2/40	GA	Y	241.000000	0.893821	6.637168	241.000000	88.879668	98.340249
machine_advantage	15	5	73	2/40	Memetic	Y	229.000000	1.804502	1.327434	229.000000	87.947598	92.139738
Specialization hand	2	2	6	1/100	ECT	Y	3.000000	0.033191	0.000000	3.000000	100.000000	100.000000
Specialization hand	2	2	6	1/100	LS	Y	3.000000	0.073672	0.000000	3.000000	100.000000	100.000000
Specialization hand	2	2	6	1/100	SA	Y	3.000000	0.137766	0.000000	3.000000	100.000000	100.000000
Specialization hand	2	2	6	1/100	TS	Y	3.000000	0.224512	0.000000	3.000000	100.000000	100.000000
Specialization hand	2	2	6	1/100	GA	Y	3.000000	0.086872	0.000000	3.000000	100.000000	100.000000
Specialization hand	2	2	6	1/100	Memetic	Y	3.000000	0.168366	0.000000	3.000000	100.000000	100.000000
Very short ops	3	3	150	1/1	ECT	Y	50.000000	0.028553	0.000000	50.000000	100.000000	100.000000
Very short ops	3	3	150	1/1	LS	Y	58.000000	0.690069	16.000000	58.000000	86.206897	89.655172
Very short ops	3	3	150	1/1	SA	Y	56.000000	1.813036	12.000000	56.000000	89.285714	92.857143
Very short ops	3	3	150	1/1	TS	Y	58.000000	4.802624	16.000000	58.000000	86.206897	89.655172
Very short ops	3	3	150	1/1	GA	Y	62.000000	1.649672	24.000000	62.000000	80.645161	82.258065
Very short ops	3	3	150	1/1	Memetic	Y	63.000000	3.366909	26.000000	63.000000	79.365079	84.126984
Near-total flex	3	3	9	10/10	ECT	Y	30.000000	0.038239	0.000000	30.000000	100.000000	100.000000
Near-total flex	3	3	9	10/10	LS	Y	30.000000	0.132600	0.000000	30.000000	100.000000	100.000000
Near-total flex	3	3	9	10/10	SA	Y	30.000000	0.227906	0.000000	30.000000	100.000000	100.000000
Near-total flex	3	3	9	10/10	TS	Y	30.000000	0.478836	0.000000	30.000000	100.000000	100.000000
Near-total flex	3	3	9	10/10	GA	Y	30.000000	0.155935	0.000000	30.000000	100.000000	100.000000
Near-total flex	3	3	9	10/10	Memetic	Y	30.000000	0.360694	0.000000	30.000000	100.000000	100.000000
Near-zero flex	3	3	9	10/10	ECT	Y	30.000000	0.033554	0.000000	30.000000	100.000000	100.000000
Near-zero flex	3	3	9	10/10	LS	Y	30.000000	0.113799	0.000000	30.000000	100.000000	100.000000
Near-zero flex	3	3	9	10/10	SA	Y	30.000000	0.195269	0.000000	30.000000	100.000000	100.000000
Near-zero flex	3	3	9	10/10	TS	Y	30.000000	0.415501	0.000000	30.000000	100.000000	100.000000
Near-zero flex	3	3	9	10/10	GA	Y	30.000000	0.183733	0.000000	30.000000	100.000000	100.000000
Near-zero flex	3	3	9	10/10	Memetic	Y	30.000000	0.399067	0.000000	30.000000	100.000000	100.000000
One job	1	5	10	5/5	ECT	Y	50.000000	0.030994	0.000000	50.000000	20.000000	100.000000
One job	1	5	10	5/5	LS	Y	50.000000	0.130378	0.000000	50.000000	20.000000	40.000000
One job	1	5	10	5/5	SA	Y	50.000000	0.246452	0.000000	50.000000	20.000000	40.000000
One job	1	5	10	5/5	TS	Y	50.000000	0.490905	0.000000	50.000000	20.000000	40.000000
One job	1	5	10	5/5	GA	Y	50.000000	0.191110	0.000000	50.000000	20.000000	40.000000

Case	J	M	N	$p_{\min}/p_{\max}$	Solver	Valid	$C_{\max}$	Time (s)	Gap (%)	CP	$\bar{U}$ (%)	$U_{\max}$ (%)
One job	1	5	10	5/5	Memetic	Y	50.000000	0.492245	0.000000	50.000000	20.000000	40.000000
Single machine	5	1	10	10/15	ECT	Y	125.000000	0.045328	0.000000	125.000000	100.000000	100.000000
Single machine	5	1	10	10/15	LS	Y	125.000000	0.119623	0.000000	125.000000	100.000000	100.000000
Single machine	5	1	10	10/15	SA	Y	125.000000	0.230418	0.000000	125.000000	100.000000	100.000000
Single machine	5	1	10	10/15	TS	Y	125.000000	0.410229	0.000000	125.000000	100.000000	100.000000
Single machine	5	1	10	10/15	GA	Y	125.000000	0.161821	0.000000	125.000000	100.000000	100.000000
Single machine	5	1	10	10/15	Memetic	Y	125.000000	0.371772	0.000000	125.000000	100.000000	100.000000
small	6	3	20	6/10	ECT	Y	83.000000	0.029897	0.000000	83.000000	67.469880	100.000000
small	6	3	20	6/10	LS	Y	83.000000	0.238393	0.000000	83.000000	67.469880	100.000000
small	6	3	20	6/10	SA	Y	83.000000	0.451106	0.000000	83.000000	67.469880	100.000000
small	6	3	20	6/10	TS	Y	83.000000	0.997140	0.000000	83.000000	67.469880	100.000000
small	6	3	20	6/10	GA	Y	83.000000	0.325828	0.000000	83.000000	67.469880	100.000000
small	6	3	20	6/10	Memetic	Y	83.000000	0.775598	0.000000	83.000000	67.469880	100.000000

6.3 Per-experiment characteristics and best observed result

Table 7 reports instance size and the best schedule among the six solvers. “Feasibility” is the number of valid schedules divided by six. All tied best methods are listed. Runtime is the fastest run among those tied-best methods; the per-algorithm table should be used for any other runtime comparison.

Table 7. Instance characteristics and best validated result per case. Runtime belongs to the fastest solver among tied best methods; case labels are shortened for layout.

Case	J	M	N	Mean flex.	$p_{\min}/p_{\max}$	Valid	Best method(s)	$C_{\max}$	LB	Gap (%)	Time (s)
average	15	5	83	0.460	3/30	6/6	SA	203	104	95.19	1.167
balanced	15	5	92	0.357	3/50	6/6	TS	329	275	19.64	3.291
bottleneck	15	5	74	0.419	2/40	6/6	LS	244	159	53.46	0.705
Bottleneck hand	10	5	30	0.600	5/50	6/6	ECT/LS/SA/TS/GA/Memetic	510	500	2.00	0.043
extreme	20	5	200	0.303	1/100	6/6	SA/TS/Memetic	1087	1087	0.00	2.957
Time gaps	3	2	9	1.000	1/10000	6/6	ECT/LS/SA/TS/GA/Memetic	9	3	200.00	0.032
High flexibility	15	5	75	0.835	2/30	6/6	TS	147	102	44.12	2.733
High variance	15	5	83	0.460	3/100	6/6	SA	586	336	74.40	1.228
Identical times	5	5	20	1.000	10/10	6/6	ECT/LS/SA/TS/GA/Memetic	40	40	0.00	0.032
large	50	10	242	0.447	2/30	6/6	SA/TS	323	110	193.64	3.831
Long chains	2	5	40	1.000	5/5	6/6	ECT/LS/SA/TS/GA/Memetic	100	100	0.00	0.034
low_flexibility	15	5	81	0.202	4/30	6/6	LS/SA/TS/GA/Memetic	419	419	0.00	0.625
machine_advantage	15	5	73	0.452	2/40	6/6	SA	226	226	0.00	1.108
Specialization hand	2	2	6	1.000	1/100	6/6	ECT/LS/SA/TS/GA/Memetic	3	3	0.00	0.033
Very short ops	3	3	150	1.000	1/1	6/6	ECT	50	50	0.00	0.029
Near-total flex	3	3	9	1.000	10/10	6/6	ECT/LS/SA/TS/GA/Memetic	30	30	0.00	0.038
Near-zero flex	3	3	9	0.333	10/10	6/6	ECT/LS/SA/TS/GA/Memetic	30	30	0.00	0.034
One job	1	5	10	1.000	5/5	6/6	ECT/LS/SA/TS/GA/Memetic	50	50	0.00	0.031
Single machine	5	1	10	1.000	10/15	6/6	ECT/LS/SA/TS/GA/Memetic	125	125	0.00	0.045
small	6	3	20	0.333	6/10	6/6	ECT/LS/SA/TS/GA/Memetic	83	83	0.00	0.030

6.4 Utilization and critical path

For every one of the 120 valid schedules, the computed resource-constrained critical path length equals its makespan. The “very short operations” and symmetric hand-built cases often use all machines heavily; by contrast the hand-built mandatory bottleneck case has low mean utilization across all machines because 50-unit forced operations dominate a 510-unit horizon while parallel machines do little work. The largest per-machine utilization in each schedule is available in the CSV; it reaches 100% in numerous cases, consistent with a critical machine arc.

The utilization figures should not be interpreted as a global efficiency objective: an algorithm can increase average machine utilization while worsening makespan, or leave machines idle to protect a critical operation. Per-run utilization and the full critical-operation sequence are retained in the CSV to support follow-up analysis.

6.5 Lower-bound results

Twelve of the twenty best observed schedules meet the lower bound defined in Section 5: extreme, identical processing times, long critical chains, low flexibility, generated machine advantage, hand-built machine advantage, many very short operations, near-total flexibility, near-zero flexibility, one job, single machine, and small. Because these are feasible schedules and the bound is valid, equality certifies an optimum for those specific constructed instances.

The remaining eight best schedules have positive gaps. The largest best-schedule gap in the table is 200% on `extreme_time_gaps`; the current bound sees a minimum 3-unit chain but does not capture the shared competition for the fast machine. This is a bound weakness, not evidence that the solvers are poor or that their schedules are far from optimal. A stronger assignment-relaxation bound is needed for cases with many shared alternatives.

6.6 Runtime and convergence

Across the 20-case matrix, mean makespans were: Greedy ECT 281.6, LS 240.4, SA 231.8, TS 231.8, GA 248.8, and Memetic 242.3. Mean runtimes in seconds were 0.036, 0.525, 1.013, 2.370, 0.913, and 1.878, respectively. These are arithmetic means over intentionally heterogeneous cases, one run per case, on one machine. They are not time-normalized or statistically reliable rankings.

No solver currently emits an iteration-by-iteration curve. Therefore this report cannot empirically say which method converged early, stagnated late, or crossed a worse basin at what iteration. Final makespan and final runtime must not be mislabeled as convergence evidence.

7 Part D Failure Analysis

The term “failure” below includes a valid schedule that is structurally weak relative to another method or a proven lower bound; it does not imply invalid output. All 120 schedules were feasible. Each analysis separates observed evidence from causal hypotheses and proposed follow-up experiments.

7.1 Irreversible ECT commitments under contention

Failure mode 1 — fast local choices create global queuing

Observation. Greedy ECT returns substantially worse makespans than the best metaheuristic on the generated average, bottleneck, low-flexibility, high-variance, and extreme cases.

Evidence. Average: ECT 242 versus SA 203 (19.2% higher than the best). Generated bottleneck: 331 versus LS 244 (35.7%). Low flexibility: 499 versus 419 (19.1%). High variance: 820 versus 586 (39.9%). Extreme: 1375 versus 1087 (26.5%). Yet on the deliberately forced hand bottleneck, all methods return 510.

Hypothesis. ECT selects the smallest immediate finish and cannot revise an assignment after future operations reveal that the chosen machine is a shared bottleneck.

Structural explanation. The current rule weighs A_m and p_{om} only for the ready operation. It omits downstream job work, the number of competing operations eligible for that machine, and whether delaying this operation blocks a critical chain. Low flexibility limits escape routes; high variance makes one misrouted long operation especially costly. The hand-built instance instead has 10 forced 50-unit operations and a 500-unit valid machine-load bound: all methods encounter the same resource floor.

Proposed improvement. Add a load-aware dispatch score combining ECT, residual job work, eligible-machine scarcity, and projected machine workload. Compare it to baseline under the same instances and report both runtime and LB gap. Keep the existing deterministic ECT rule as a baseline.

7.2 High flexibility increases routing opportunity and coupling

Failure mode 2 — more options enlarge assignment search

Observation. The high-flexibility case is not automatically easier; the quality ranking spreads while all schedules remain valid. TS obtains 147, versus LS 159 and SA 160; GA is at 191.

Evidence. Mean eligibility is 0.835 of machines in the high-flexibility instance, versus 0.202 in low flexibility. On high flexibility, best makespan is 147 and LB is 102 (44.1% gap). On low flexibility, all five metaheuristics reach the 419 lower bound; the greedy baseline is the exception at 499.

Hypothesis. High flexibility creates a larger combinatorial machine-assignment space. Random machine-map edits can move operations to individually fast machines that collectively overload one resource; TS’s 20-neighbor sample can find useful coordinated changes more

often in this one fixed run.

Structural explanation. An operation's machine choice changes both its duration and the disjunctive machine-order arcs. The OS and machine-map components are coupled: a routing improvement may only pay if sequencing also changes. More choices reduce forced workload and can weaken the current lower bound, so the remaining gap can increase even as the problem becomes more flexible.

Proposed improvement. Use machine-load-aware machine-map neighborhoods and critical-machine moves; add diversity measures for machine assignments. Repeat over multiple algorithm seeds before attributing the high-flexibility outcome to TS.

7.3 Long-duration variance and non-scaled SA temperature

Failure mode 3 — high variance makes fixed search parameters scale-sensitive

Observation. The high-variance gap between baseline and metaheuristics is large, while the fixed-budget methods still return distinct final scores.

Evidence. High variance: ECT 820, LS 697, SA 586, TS 607, GA 680, and Memetic 638. The best score improves 28.5% over ECT; SA is 3.5% better than TS on this run. The extreme case uses 200 operations and the best schedule equals the 1087 lower bound; the mean instance has an LB gap of 95.2%.

Hypothesis. A single absolute SA starting temperature does not scale with the distribution of uphill deltas. Random local edits may also miss the small number of long, critical operations that determine the horizon.

Structural explanation. Long operations often become vertices on the critical path or dominant machine arcs. Moving a noncritical short operation changes little; a single assignment change on a long critical operation can dominate the objective. At larger objective scales, the same Δ/T acceptance probability becomes small, suppressing escape moves.

Proposed improvement. Estimate a starting temperature from sampled positive deltas to target an initial acceptance rate, log temperature/acceptance/best-so-far, and compare critical-path-weighted move sampling against uniform sampling.

7.4 Many very short operations expose loss of a strong incumbent

Failure mode 4 — metaheuristics can finish worse than a one-pass baseline

Observation. On the 150-operation, 1-unit many_very_short_operations instance, every metaheuristic returns a worse makespan than ECT.

Evidence. ECT obtains 50, matching the valid LB. LS returns 58, SA 56, TS 58, GA 62, and Memetic 63; every schedule is feasible. The critical path is exactly 50.

Hypothesis. Search initializes from random OS/machine-map states rather than the greedy incumbent, and none of the solvers injects ECT into its starting state/population. The search

budget can spend many evaluations in a state whose objective is already no better than the easy-to-compute baseline.

Structural explanation. With equal 1-unit durations and high flexibility, assignment quality has little value while sequence/load balance dominates. The lower bound is tight and the baseline immediately constructs a schedule meeting it. Random swaps and population recombination can disrupt a balanced schedule; best-so-far preservation only preserves candidates actually evaluated.

Proposed improvement. Seed LS, SA, and TS with the ECT schedule; place ECT plus small perturbations into the Genetic and Memetic populations. Enforce elitism against the incumbent and stop when a valid lower bound is met. Compare the extra initialization cost to final quality.

7.5 Weak lower bounds on shared fast alternatives

Failure mode 5 — reported optimality gap can be a bound artifact

Observation. All six solvers return 9 on the extreme time-gap case, while the simple bound is only 3; the table shows a 200% gap even though the schedule outcomes are identical.

Evidence. Three jobs, two machines, nine operations, each with a 1-unit versus 10,000-unit alternative. Every schedule is feasible and has makespan 9. The job-chain bound is 3; no operation has a single eligible machine, so the forced-machine term contributes zero.

Hypothesis. The high numeric gap is driven primarily by a relaxation that omits collective resource competition, not by evidence of a bad candidate.

Structural explanation. The bound sums per-operation minima independently per job. It does not recognize that selecting the 1-unit option sends many operations to the same fast machine. Conversely, selecting the slow alternative is disastrous. The assignment choices are globally coupled by machine capacity.

Proposed improvement. Add a machine-assignment relaxation that minimizes maximum machine load subject to eligibility/durations, or exact branch-and-bound for tiny cases. Label the present quantity a simple lower bound and never infer quality solely from its gap.

7.6 Flat landscapes in deterministic edge cases

Failure mode 6 — search effort cannot improve a structural floor

Observation. All methods tie on single machine (125), one job (50), long chains (100), near-zero flexibility (30), near-total flexibility (30), identical times (40), and hand specialization (3); 12 cases meet the lower bound.

Evidence. In every case listed, all six independent schedules are feasible and the best result equals the stated lower bound. In several, all six methods return exactly the same makespan.

Hypothesis. These constructions remove or neutralize one or both search decisions: there is one resource, one precedence chain, only one eligible machine, equal-duration alternatives, or a clear specialization with a tight bound.

Structural explanation. OS/machine-map variation either has no effect on durations/resources or cannot beat the precedence/mandatory-machine floor. Objective ties make a stochastic search appear active without changing makespan. Runtime is then spent rediscovering a known structural optimum.

Proposed improvement. Detect degenerate structure from eligibility and duration symmetry; use a fast exact/simple construction or stop when the valid lower bound is met. Preserve these cases as regression tests rather than using them alone to rank stochastic methods.

7.7 Population methods scale their evaluation cost

Failure mode 7 — larger search population did not buy a better large-case result

Observation. On the 50-job/10-machine case, GA and Memetic lag both SA and TS and use more runtime than LS.

Evidence. Large case: SA and TS each return 323; LS returns 343; GA 434; Memetic 426. Recorded runtimes are 3.83 s (SA), 8.78 s (TS), 3.82 s (GA), and 7.30 s (Memetic). All outputs are feasible. Best LB is 110, so the remaining gap is still large at 193.6%.

Hypothesis. Random initialization plus fixed population/generation budgets fails to intensify around high-quality states quickly enough. In the Memetic implementation, local search multiplies candidate evaluations while the source has no chromosome mutation or diversity control; this combination is a plausible, untested explanation, not a proven cause.

Structural explanation. At 242 operations, the combined sequencing and assignment space is large. A population generation evaluates many full candidates; the memetic offspring additionally incur local-search evaluations. The lower bound is weak for flexible alternatives, so the large gap cannot distinguish poor search from an optimistic/weak bound.

Proposed improvement. Use an ECT-seeded population, adaptive mutation/diversity control, and selective local search. Benchmark under equal wall-clock and equal evaluator-call budgets; record anytime curves and ablate the local-search component.

7.8 No observed feasibility failure; remaining untested failures

No invalid schedule, timeout, or crash occurred in the 120-run matrix. The validator caught no violation, so this dataset provides no measured example of a precedence, machine-overlap, or eligibility failure. It does not test malformed JSON, duplicate operation IDs in source instances, NaN/Infinity JSON, unsupported CLI flags, or empty eligible-machine arrays. Those need directed negative tests; absence of failure in this matrix is not evidence that such inputs are safe.

7.9 Source-level failure analysis: behavior not isolated by the benchmark

The following findings come from execution order and operator inspection in the current source. They are not presented as causes of the measured makespans: the experiment matrix has no counters or controlled ablations that would establish that causal link.

Source finding 8 — tabu aspiration is evaluated after the best value changes

Observation. The Tabu Search source labels a global-best aspiration rule, but the predicate cannot identify a newly best candidate as a global best.

Evidence. `eval.evaluate(neighbor_sol)` runs before `neighbor_makespan < eval.get_best_objective()` is computed. For a valid strict improvement, the evaluator has already set its best objective to that candidate's makespan. The comparison is then equality and returns false.

Hypothesis. The aspiration condition was designed assuming the evaluator only scores candidates, while it also mutates best-so-far state.

Structural explanation. The solver's admissibility decision uses global-best state as an exception to tabu membership. Updating shared best-so-far state during scoring changes the value against which that exception is tested; candidate evaluation and move admissibility are therefore coupled in the wrong order.

Proposed improvement. Snapshot the global best before evaluation or have evaluation return a score without mutating search policy state. Add a test where a tabu-marked candidate strictly improves the saved best and assert that the aspiration route admits it.

Source finding 9 — the memetic solver has no chromosome mutation

Observation. The implemented Memetic Algorithm varies offspring through crossover and local-search proposals, but not through the Genetic Algorithm's 10% operation-sequence/machine-map mutation.

Evidence. `memetic_algorithm.cpp` defines crossover and local-search methods and calls both when producing offspring; it does not define or call a chromosome mutation method.

Hypothesis. The design was intended to share the genetic operators, but only its representation, selection, crossover, and elitism were carried over when the hybrid local-search step was added.

Structural explanation. If the population loses variation in job order or machine assignment, crossover between similar parents may not restore it. Strict-improvement local search can intensify members within their current basins, while refusing neutral and uphill steps that might escape those basins. This mechanism is plausible, not proven to explain the large-case result.

Proposed improvement. Decide explicitly whether mutation belongs in the memetic design. If added, compare a mutation-enabled ablation with the current implementation under matched evaluator-call and wall-clock budgets; record population diversity and per-generation best-so-far values.

8 Project Readiness and Reproducibility Gaps

Table 8. Prioritized follow-up actions grounded in the reviewed code and measurements.

Priority	Area	Required work
P0	Regression tests	Add tests for OS multiplicities, machine-map eligibility, precedence, machine overlap, finite times, exact small-case LB equality, tabu aspiration, and all hand-built edge files.
P0	Reproducibility	Expose algorithm seed and search budget; write seed, compiler, flags, revision, instance hash, solver settings, host, and run status to a manifest. Current solver seed 42 is compiled into source.
P0	Search semantics	Correct Tabu Search's aspiration check ordering and canonicalize its whole-state key/expiry behavior. Explicitly decide whether Memetic Algorithm should mutate; test the chosen operator set rather than describing it as GA-identical.
P1	Convergence evidence	Emit best-so-far makespan, current makespan, accepted moves, temperature, tabu skips, population diversity, evaluator-call count, and elapsed time at configurable checkpoints.
P1	Independent result handling	Keep the runner's validation-before-scoring behavior. Add validator-version/error columns and preserve logs. Add malformed-instance and malformed-schedule negative tests.
P1	Bound quality	Implement and validate an assignment/load relaxation for flexible shared alternatives; compare small instances against exhaustive or exact search. Preserve the simple bound as a fallback with clear naming.
P1	Fair algorithm budgets	Compare equal wall-clock and equal evaluator-call budgets, plus a separate equal-iteration study. Publish per-instance paired outcomes and variability over multiple seeds.
P1	Data provenance	Record generator version, full parameters, instance hash, and seed. The current generated batches are reproducible only with the same C++ standard-library/toolchain behavior.
P2	Large-scale path	Use indexed compact integer IDs and avoid copying hash maps/strings on every proposal. Profile before adding incremental evaluator complexity. The 1,000-job JSON is currently a generator stress case, not part of the six-solver matrix.
P2	Build and portability	Replace manual platform-specific executable commands with a repeatable build target; make experiment discovery and executable suffix platform-aware; remove the stale absolute compiler path in <code>.vscode/c_cpp_properties.json</code> .

8.1 What this report does not establish

One run per instance and one hard-coded seed cannot estimate robustness, confidence intervals, or statistical significance. The average runtime mixes algorithms with different candidate-evaluation budgets. The 1,000-job/50-machine generator output was regenerated, but it was not included in the 120 algorithm runs. Historical runtime values should not be compared to the new values without matching source revision, compiler, hardware, and run setup.

9 Source-by-Source Implementation Audit ---

This section records the implementation choices that connect the design descriptions to the files that execute them. It is not a claim that every line in the vendored JSON library has been re-reviewed. First-party generator, solver, validation, experiment, and visualization modules are covered; the prebuilt executables and generated JSON files are treated as artifacts and inputs, not as independent implementations.

9.1 Generator and instance-file path

Table 9. Generator-side modules and consequential implementation decisions.

Source	Implemented decision	Consequence or limit
<code>generator/gen_core.cpp</code>	Owns CLI parsing, preset application, generation order, role selection, job weights, operation counts, eligibility, processing-time transformations, metadata assembly, post-construction validation, and output selection. Class presets override class-level fields while preserving jobs, machines, and seed. Without a class, core parameters and seed must be provided; extended parameters are filled from defaults.	The shared seeded draw order couples downstream samples to earlier choices. Only job/machine counts and processing-time endpoints receive final range checks; several probability and cross-field ranges remain unchecked. Unknown processing-time distribution names reach the uniform fallback in the distribution wrapper. An unsupported <code>-format</code> value is not rejected; it can leave the process successful without writing a recognized format. In both mode, an output path receives JSON and a sibling <code>.txt</code> text file; without a path both representations are printed to standard output with a separator.
<code>generator/distributions.h</code>	Wraps <code>std::mt19937_64</code> in one <code>DistEngine</code> ; provides normal, log-normal, Poisson, gamma, Bernoulli, integer/real uniform, categorical, and partial Fisher–Yates subset sampling.	Randomness is centralized and seeded, but standard-library distributions are not specified to produce identical streams across vendors or versions. The categorical helper normalizes weights; its callers still need valid nonnegative inputs.

Source	Implemented decision	Consequence or limit
<code>generator/fjsp_instance.h</code>	Stores jobs, ordered operations, eligible machine-time pairs, roles, and metadata. Internal job/machine indices are one-based and operation indices zero-based; output IDs are padded at serialization. JSON and Brandimarte-style text are emitted by hand.	Structural checks catch empty jobs/operations, noncontiguous job IDs, operation ownership/index mismatches, empty or duplicate eligibility, out-of-range machine IDs, nonpositive durations, and total-operation mismatch. Hand-written JSON strings require correct escaping when new untrusted string fields are introduced.
<code>generator/gen_orchestrator.py</code>	Finds a compiled <code>gen_core</code> binary, wraps single/batch/stats commands, names batch files by class/scale/seed, optionally calls the dashboard visualizer, and reports failed subprocesses.	It orchestrates rather than independently validates generated instances. Its Python single-instance option set forwards only a subset of the C++ core's extended parameters; direct C++ invocation is required for all custom fields. Batch output directories can contain prior files not overwritten by the current requested class/seed set.
<code>generator/gen_visualizer.py</code>	Loads instance metadata and produces a multi-panel view of processing-time distributions, operations/job, flexibility, eligible load by machine, job weights, per-job times, machine/job eligibility, and machine-role times.	Matplotlib is optional. The visualizer is diagnostic only: a rendered chart does not establish instance validity, schedule feasibility, or solver quality.

Generation guarantees are conditional on valid parameter ranges and a successful call to the instance validator. The output model makes job order explicit through operation-array order; it does not store arbitrary precedence arcs. The class names summarize preset settings, not statistical guarantees about every generated realization.

9.2 Shared solver representation and independent checks

Table 10. Solver infrastructure, validation boundary, and saved-output rules.

Source	Implemented decision	Consequence or limit
<code>algorithms/fjsp_algorithm.h</code>	Loads JSON into job/operation structures and builds an operation-ID-to-index lookup. Decodes OS occurrences in one pass and builds each schedule by advancing job and machine availabilities to the earliest shared free time.	Expected constant-time indexed operation lookup removes a repeated full instance scan. The decoder does not insert an operation into a previously created idle gap; its representable schedules are restricted by serial decoding.
<code>environment/evaluator.cpp</code>	Loads instance lookups once, validates each explicit candidate, returns infinity with diagnostics on invalid candidates, and stores a full copy only when a strictly smaller makespan is found. Schedule export is delayed until the best is requested and sorts by start time with operation ID as tie-break.	The evaluator's best state is separate from an algorithm's current state. Equal makespans do not replace the stored best. Full validation and machine timeline sorting are repeated for every candidate.
<code>environment/validator.cpp</code>	Reads instance and schedule independently. It checks schedule shape, ID types and ownership, completeness and duplicates, finite/nonnegative times, machine eligibility and duration, job precedence, and machine overlaps; only valid schedules receive a makespan.	This is the experiment acceptance authority, separate from solver scoring. Foundational errors stop later phases. It assumes the parsed instance has a usable expected schema; malformed-instance hardening and negative tests remain unverified.
<code>environment/json.hpp</code>	Vendored nlohmann JSON single-header dependency used by the C++ reader and writer.	Avoids a separate C++ package-install step. It is a dependency, not a project-authored generator, solver, or validation module.

9.3 Solver entry points

Every solver executable parses `-instance` and optionally `-output`; the stochastic solvers do not parse a seed option. They include the evaluator implementation with its standalone main disabled, print an evaluator summary, and export the saved best only when an output path is supplied. The implementation details of their state transitions, exact settings, evaluation counts, and limitations are given in Section 4; this source index identifies the responsibility of each file.

Table 11. First-party scheduling executables.

Source	Main responsibility	Important source-level boundary
<code>algorithms/greedy_spt.cpp</code>	Repeated earliest-completion choice across ready job operations and eligible machines; advances one job and one machine after each choice.	Despite its file-name/comment, its objective is ECT, not global SPT. Strict comparison leaves exact ties to iteration order.
<code>algorithms/local_search.cpp</code>	One random OS/machine-map state; 10,000 random swap or reassignment proposals; accepts valid non-worsening neighbors.	The neighborhood can produce no-ops. Neutral acceptance is enabled here, while the memetic polishing routine uses strict improvement.
<code>algorithms/simulated_annealing.cpp</code>	Uses the LS move family with probabilistic uphill acceptance and geometric cooling.	The absolute start temperature and cooling factor are fixed; there is no acceptance-rate or convergence instrumentation.
<code>algorithms/tabu_search.cpp</code>	Samples 20 one-move candidates per iteration, chooses a best admissible candidate, and maintains a tenure-100 queue/set of full-state strings.	Aspiration is checked after evaluator best-state update, making its strict new-best predicate ineffective. The key is not canonicalized and the run has no fallback if all sampled candidates are tabu.
<code>algorithms/genetic_algorithm.cpp</code>	Uses size-two tournament, one elite, uniform per-operation machine-map crossover, job-based OS crossover, and independent 10% OS/machine-map mutation probabilities.	No greedy individual, repair, restart, adaptive mutation, or explicit diversity measure is implemented.

Source	Main responsibility	Important source-level boundary
<code>algorithms/memetic_algorithm.cpp</code>	Uses a smaller GA-style population with 50 local-search proposals per initial member and 10 per offspring; local polishing accepts strict improvement.	The source has no GA mutation operator or mutation call. Its evaluation budget is several times the GA budget, so raw quality is not an equal-effort comparison.

9.4 Experiment, visualization, and publication scripts

Table 12. Python and data-package modules used around the C++ components.

Source or artifact	Implemented responsibility	Interpretation and limit
<code>experiments/hand_builder.py</code>	Constructs ten named structural edge cases and serializes them as readable JSON with explicit eligible machine-time lists.	These fixtures complement generated cases; they target fixed routing, precedence chains, forced contention, extreme durations, symmetric choices, and many short operations. The checked-in JSON files are the runner inputs.
<code>environment/part_d_runner.py</code>	Sorts matching instance JSON files; removes each old target schedule; runs six solver executables with a per-process timeout; validates outputs using the independent C++ validator; then derives instance, schedule, lower-bound, critical-path, and utilization fields. It writes one CSV row per instance/solver and a diagnostic log for failures.	Runtime uses a monotonic performance clock and excludes the validator. Validation failure, timeout, missing output, or parse/metric error is recorded as a failed row rather than scored as a feasible schedule. The default experiment input folder is <code>experiments/</code> .
<code>environment/metrics.py</code>	Retained earlier experiment driver: invokes the six algorithms, reads completion times from schedule JSON, writes a compact makespan/runtime CSV, and creates Gantt images.	This driver does not invoke the independent validator and uses a 60-second timeout. It is not the source of the current 120-row Part D evidence; use <code>part_d_runner.py</code> for the reported matrix.

Source or artifact	Implemented responsibility	Interpretation and limit
environment/export_part_d_latex.py	Copies the Part D CSV into the Overleaf data directory and emits <code>per_run_detail.tex</code> and <code>case_summary.tex</code> . It groups by case, lists all tied best algorithms, and reports the fastest runtime among those tied-best methods in the case summary.	The exported tables are generated artifacts. The source CSV remains the primary full-precision experiment record; the LaTeX tables round values for print layout.
algorithms/visualize.py	Draws a schedule Gantt chart with machines as rows, time as the horizontal axis, and job-specific colors; operation/job IDs appear inside bars.	It renders schedule JSON as supplied and does not independently validate the schedule. Its machine sorting assumes IDs contain a numeric suffix.
docs/report/data/part_d_run_matrix.csv	Packaged copy of <code>analysis/part_d_results.csv</code> , alongside generated LaTeX tables for inclusion in the report.	The copy matched the analysis CSV at the last package check. Re-run the exporter after changing the experiment CSV.
.vscode/c_cpp_properties.json	Declares C17/C++17 and Windows GCC IntelliSense settings for the editor.	The compiler path is an absolute machine-specific path (<code>C:/NEW_DOWNLA0DS/mingw/mingw64/bin/gcc</code>); it is not portable and appears stale for the reviewed check-out. This editor setting does not build the project or prove that the report's documented commands work.

9.5 Documentation and project boundary

`README.md` documents repository setup and command examples; `docs/GENERATOR_DESIGN.md` and `docs/PROJECT_ASSUMPTIONS.md` retain focused source-grounded developer notes. The current requirements file declares Matplotlib for visualization; the generator orchestrator otherwise uses the Python standard library. `requirements.txt` pins Matplotlib to 3.9.0; no scheduling or generation Python package is declared. The Overleaf project has its own preamble, settings, section sources, and generated data tables, so it does not import repository files at compile-time.

Documentation discrepancy retained as a readiness item

The root README mentions a Python environment/validator.py and describes it as runnable, but no such file is present in the reviewed environment/ directory. The implemented independent validator is environment/validator.cpp. This report follows the checked-in implementation and does not treat the README's Python-validator reference as evidence of a second validator.

The two long single-file reports under LEFT/ are style and narrative references, not dependencies of this Overleaf package. Their claims were not copied automatically: report statements are checked against the current sources and current Part D data, and only files under docs/report/ are needed to compile the report.

10 Reproducibility and Supplementary Definitions

The Overleaf project contains the complete report sources, figures, and packaged Part D tables; compiling the PDF does not require a local checkout of the repository. Re-running the experiments does require the repository root, a C++17 compiler, and Python. The following PowerShell commands reproduce the current native executables used by the generator and Part D runner:

```
g++ -O3 -std=c++17 generator/gen_core.cpp -o generator\gen_core.exe
g++ -O3 -std=c++17 algorithms/greedy_spt.cpp -o algorithms\greedy_spt.exe
g++ -O3 -std=c++17 algorithms/local_search.cpp -o algorithms\local_search.exe
g++ -O3 -std=c++17 algorithms/simulated_annealing.cpp '
-o algorithms\simulated_annealing.exe
g++ -O3 -std=c++17 algorithms/tabu_search.cpp -o algorithms\tabu_search.exe
g++ -O3 -std=c++17 algorithms/genetic_algorithm.cpp '
-o algorithms\genetic_algorithm.exe
g++ -O3 -std=c++17 algorithms/memetic_algorithm.cpp '
-o algorithms\memetic_algorithm.exe
g++ -O3 -std=c++17 environment/validator.cpp -o environment\validator.exe
```

To regenerate the two generator batch collections (this overwrites their existing JSON outputs):

```
python generator/gen_orchestrator.py batch --jobs 10 --machines 5 '
--seeds 1,2,3 --output-dir instances/batch
python generator/gen_orchestrator.py batch --jobs 20 --machines 8 '
--seeds 1,2,3 --output-dir instances/batch_test
```

The Part D runner reads its curated inputs from experiments/; it does not benchmark the two generated batches above. Run the 20-case experiment matrix and refresh the packaged Overleaf tables with:

```
python environment/part_d_runner.py --timeout 180
python environment/export_part_d_latex.py
```

The runner writes schedules to `results/part_d/`, a full run table to `analysis/part_d_results.csv`, and diagnostics for failed runs to `analysis/part_d_logs/`. The exporter packages the current CSV as `docs/report/data/part_d_run_matrix.csv` and generates the per-run/per-case LaTeX tables. A successful solver row is not marked feasible until the independent C++ validator accepts its output. Timings are host- and build-specific; solver seeds and budgets remain compiled into the current sources.

10.1 Per-Run Data Dictionary

Field	Meaning
<code>instance</code> , <code>instance_type</code> , <code>purpose</code>	Scenario identity and intended structural stress.
<code>algorithm</code> , <code>algorithm_seed</code>	Solver name; source-level fixed default seed (Greedy is deterministic).
<code>feasible</code> , <code>diagnostic</code>	Independent validator status and failure reason.
<code>makespan</code> , <code>runtime_seconds</code>	Objective for valid output and solver wall time.
<code>jobs</code> , <code>machines</code> , <code>operations</code>	Instance scale counts.
<code>mean_flexibility</code>	Mean eligible alternatives divided by machine count.
<code>processing_time_min</code> , <code>processing_time_max</code> , <code>processing_time_mean</code> , <code>processing_time_stddev</code>	Range and population statistics over all eligible machine-duration entries.
<code>bottleneck_machines</code> , <code>specialist_machines</code>	Generator role IDs when present; empty for hand-built cases without role metadata.
<code>lower_bound</code> , <code>lower_bound_gap_percent</code>	Simple valid bound and percentage gap defined in Section 5.
<code>critical_path_length</code> , <code>critical_path_operations</code>	Longest weighted path and its operation IDs in the schedule resource graph.
<code>machine_utilization_mean_percent</code> , <code>machine_utilization_max_percent</code>	Mean over all machines and maximum per-machine busy-time ratio.
<code>convergence_trace</code>	Explicit statement that the solver does not emit iteration-wise convergence data.
<code>schedule_path</code>	Path to the validated JSON output for this run.

11 Algorithm Comparison

11.1 Aggregate results from the 20-case run

Table 14. One-run arithmetic aggregates over the deliberately mixed 20-case suite.

Algorithm	Feasible	Mean $C_{\max}$	Mean runtime (s)	Strict row wins
Greedy ECT	20/20	281.60	0.036	1
Local Search	20/20	240.40	0.525	1
Simulated Annealing	20/20	231.80	1.013	3
Tabu Search	20/20	231.80	2.370	2
Genetic Algorithm	20/20	248.75	0.913	0
Memetic Algorithm	20/20	242.30	1.878	0

There are many ties, especially on edge cases whose lower bound is met by every solver. Strict wins count only a unique minimum; they do not count ties. SA and TS have equal rounded arithmetic mean makespan in this sample, but SA's mean runtime is less than half TS's. GA is faster than TS and Memetic on average, but slower than LS in this run; it loses on the generated large case and does not lead any case uniquely. Greedy is orders faster than the search methods and is the unique best method on the very-short-operation case; it also ties the lower bound on several structural cases. It is a useful incumbent, not a robust quality winner on the generated difficult cases.

11.2 Trade-off by method

Table 15. Project-specific strengths and costs; not a universal ranking.

Method	Best role	Evidence in this suite	Main trade-off
Greedy ECT	Fast base-line/incumbent	Runtime mean 0.036 s; reaches LB on many degenerate cases.	Fails to revise locally attractive but globally congesting decisions.
LS	Low-cost intensification	Mean makespan 240.4; improves average, bottleneck, high-variance, and extreme over base-line.	Cannot accept uphill moves; can lose to ECT on 150 short operations.
SA	Quality/runtime default candidate	Mean 231.8 at 1.013 s; strongest outcomes on average, high variance, extreme, and machine advantage.	Fixed temperature and uninstrumented convergence; seed sensitivity unknown.

Method	Best role	Evidence in this suite	Main trade-off
TS	Broader sampled local neighborhood	Ties best on average, balanced, extreme, large; wins high flexibility.	Highest metaheuristic runtime mean, 2.370 s; expensive 20-neighbor scan and whole-state tabu.
GA	Population diversity baseline	Feasible on all cases; lower runtime than LS in this suite.	Mean makespan 248.75; weak large-case result and no unique case win.
Memetic	Intensified population exploration	Ties the best extreme score and remains feasible throughout.	Mean runtime 1.878 s; LS effort increases cost and can erode diversity.

Because the parameter budgets differ, the aggregate table answers “what did these configured programs do on these cases?” It does not answer which search paradigm is intrinsically better under equal computational effort.

12 Recommended Solver and Decision Rule

12.1 Provisional selection

Recommendation for this codebase

Use Simulated Annealing as the provisional default when a search-based result is required; first generate a Greedy ECT incumbent and retain it as a guaranteed fallback. Offer Tabu Search as a slower alternative when the user values broad sampled neighborhoods and can afford the additional evaluations. This is a practical choice from one fixed-seed run, not a statistically proven universal winner.

The evidence for SA is the lowest rounded mean makespan (tied with TS at 231.8), about 1.013 seconds mean runtime versus TS at 2.370 seconds, and a strong result on high variance/extreme cases. TS is superior on the single high-flexibility case and balanced case; SA and TS tie on large and extreme. Case-specific results therefore matter. ECT should be inserted into every stochastic initial state/population so search cannot return a worse result than the cheap baseline.

12.2 Decision rule by operating requirement

1. **Strict latency / fallback:** run ECT and return it if the time budget expires before a validated improvement.
2. **General quality/runtime:** run seeded SA from the ECT incumbent; stop early if the certified lower bound is met.
3. **Extra quality budget:** compare TS under the same wall-clock or evaluation-call budget; do

not grant it a larger budget without recording that fact.

4. **Large instances:** benchmark runtime and memory on representative scale before enabling population/memetic methods; the checked-in 1,000-job generator case has not been solved by this suite.
5. **Research/competition claims:** repeat with independent seeds, instrument convergence, report confidence intervals/paired comparisons, and compare small cases to exact optima or stronger lower bounds.

12.3 Final caveat

The current Part D results show that the code produces feasible schedules on the tested generated and hand-built cases. They also show concrete quality failures: irreversible ECT choices on contention-heavy random instances, metaheuristics worse than a tight baseline on very short symmetric operations, and a lower-bound gap that is misleading on shared fast-machine alternatives. These observations justify the proposed engineering work, but the fixed seed and single observation per instance do not establish robust rankings. The most suitable algorithm depends on case structure, acceptable runtime, and whether a validated lower bound is available.